

Asignatura: Fundamentos de algoritmia

Evaluación continua. Cuestionario.

Grupo E.
Profesor: Isabel Pita.

Nombre del alumno:

1. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n; std::cin >> n;
for (int i = 1; i < n-1; ++i)
    for (int j = i-1; j < i+1; ++j)
        x += v[i][j];
}
```

Respuesta:

El bucle externo da $n - 2$ vueltas, siendo n el valor de entrada. Para calcular el coste del bucle nos falta comprobar si todas las ejecuciones del cuerpo del bucle tienen el mismo coste y en caso afirmativo cual es el coste de una ejecución del cuerpo del bucle.

El cuerpo del bucle es un bucle for, que da únicamente 3 vueltas y cada vuelta tiene coste constante por tratarse de una operación aritmética y de un acceso a una matriz. Por lo tanto el coste del bucle interno es $3 * 1 = 3 \in \mathcal{O}(1)$

Por lo tanto el coste aproximado del bucle externo es $n * 1 = n \in \mathcal{O}(n)$, es decir lineal en el valor de n , siendo n el valor leído de la entrada.

2. Indica la complejidad en tiempo más ajustada del siguiente algoritmo, y justifícala indicando el número de vueltas que da cada bucle y el coste de cada vuelta.

```
int n, m; std::cin >> n >> m;
int i = n;
while (i > 0) {
    for (int j = 0; j < m; ++j) x += 2;
    i = i / 2;
}
```

Respuesta:

El bucle externo da $\log n$ vueltas, siendo n el valor de entrada, ya que en cada vuelta el valor de la variable de control i se divide entre 2 (Si i comienza con el valor n y en cada vuelta se divide por dos, hay que dar $\log n$ vueltas para alcanzar el valor 0). Para calcular el coste del bucle nos falta comprobar si todas las ejecuciones del cuerpo del bucle tienen el mismo coste y en caso afirmativo cual es el coste de una ejecución del cuerpo del bucle.

El cuerpo del bucle es un bucle for y una asignación de coste constante. Como ambas instrucciones se encuentran en secuencia, el coste es el mayor de ellos que corresponde al coste del bucle for. Este bucle da m vueltas, siendo m el segundo valor de entrada. Cada vuelta de este bucle tiene coste constante por tratarse de una operación aritmética y de una asignación. Por lo tanto el coste del bucle interno es $m * 1 = m \in \mathcal{O}(m)$

Por lo tanto el coste del bucle externo es $n * m \in \mathcal{O}(n * m)$, siendo n y m los dos valores de entrada.

3. Escribe un predicado para expresar que todos los valores de un vector v son diferentes.

Respuesta:

$\text{diferentes}(v) \equiv \forall x, y : 0 \leq x < y < v.size() : v[x] \neq v[y]$.

4. Escribe un predicado para indicar que los valores de un vector v están ordenados en orden creciente.

Respuesta:

$\text{creciente}(v) \equiv \forall x : 0 \leq x < v.size() - 1 : v[x] \leq v[x + 1]$

Otra posibilidad es

$\text{creciente}(v) \equiv \forall x, y : 0 \leq x < y < v.size() : v[x] < v[y]$

5. Dada la especificación:

$P: \{ a.size() \geq 0 \wedge b.size() \geq 0 \wedge ordenado(a) \wedge ordenado(b) \wedge diferentes(a) \wedge diferentes(b) \}$
 $resolver(vector<int> a, vector<int> b) \text{ dev } vector<int> sol$
 $Q: \{ (\forall k : 0 \leq k \leq sol.size() : sol[k] \in a \wedge sol[k] \in b) \wedge (\forall k : 0 \leq k \leq a.size() \wedge a[k] \in b : a[k] \in sol) \wedge$
 $(\forall k : 0 \leq k \leq b.size() \wedge b[k] \in a : b[k] \in sol) \}$

donde *ordenado* y *diferentes* son los predicados dados anteriormente y $v[k] \in a$ es un predicado que indica que el elemento $v[k]$ pertenece al vector a .

Y dada la siguiente implementación:

```
std::vector<int> resolver(std::vector<int> const& a, std::vector<int> const& b) {
    std::vector<int> sol;
    int i = 0; int j = 0;
    while (i < a.size() && j < b.size()) {
        if (a[i] < b[j]) ++i;
        else if (a[i] > b[j]) ++j;
        else { // son iguales
            sol.push_back(a[i]);
            ++i; ++j;
        }
    }
    return sol;
}
```

Indica un invariante del bucle que nos permita probar la corrección del mismo:

Respuesta:

$I: \{ (\forall k : 0 \leq k \leq sol.size() : sol[k] \in a \wedge sol[k] \in b) \wedge (\forall k : 0 \leq k \leq i \wedge a[k] \in b : a[k] \in sol) \wedge$
 $(\forall k : 0 \leq k \leq j \wedge b[k] \in a : b[k] \in sol) \wedge 0 \leq i \leq a.size() \wedge 0 \leq j \leq b.size() \}$

6. Da una función cota que permita probar la terminación del bucle de la función anterior.

Respuesta:

La función cota debe ser positiva y decrecer en cada vuelta:

$cota(a, b, i, j) = (a.size() - i) + (b.size() - j) + 1$

7. Describe un algoritmo *óptimo* para eliminar todas las ocurrencias de un elemento de un vector ordenado con elementos repetidos, e indica su complejidad.

Respuesta:

Como el vector está ordenado, podemos realizar una búsqueda binaria que obtenga la posición más a la izquierda del elemento que queremos eliminar. Realizaremos otra búsqueda binaria que obtenga la posición más a la derecha del elemento que queremos eliminar. Ambos algoritmos tienen coste $\mathcal{O}(\log n)$, siendo n el número de elementos del vector.

Desplazamos los elementos que quedan a la derecha de los elementos a eliminar tantos elementos a la izquierda como elementos a eliminar haya. Este proceso tiene coste lineal respecto al número de elementos del vector en el peor caso, ya que si se nos pide eliminar el elemento menor y este solo aparece una vez, habrá que desplazar todos los valores del vector menos el que se elimina. Sin embargo, el coste puede ser mejor si los valores de los elementos a eliminar son altos o están muchas veces repetidos.

Por último se redimensiona el vector eliminando tantos elementos como elementos iguales al elemento a eliminar hubiera en el vector. Como se eliminan los elementos del vector situados más a la derecha esta operación tiene coste constante si la eliminación del tipo de datos del vector se puede realizar en tiempo constante.

8. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del algoritmo de búsqueda binaria. Haz el despliegue e indica el coste final del algoritmo.

Respuesta:

En el algoritmo de búsqueda binaria se realiza una única llamada recursiva con la mitad de los datos de entrada. El coste de cada llamada es constante porque las operaciones que se realizan sin contar las llamadas recursivas son todas de coste constante.

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ T(n/2) + c_1 & \text{if } ini < fin \end{cases}$$

El desplegado y el coste final se encuentran en el cuaderno de problemas de complejidad de algoritmos recursivos del campus.

9. Indica cuál es la recurrencia que permite calcular el coste en el caso peor del siguiente algoritmo y el coste final del algoritmo.

```
int f ( std::vector<int> & v, int ini, int fin){
    if (ini == fin) return 0; // Vector vacio
    else {
        int m = (ini + fin - 1) / 2;
        int x1 = f(v, ini, m+1);
        int x2 = f(v, m+1, fin);
        return x1 + x2;
    }
}
```

Respuesta:

Se realizan dos llamadas recursivas, cada una con la mitad de los elementos. El coste de cada llamada es constante porque todas las instrucciones son asignaciones y sólo se realizan operaciones aritméticas si no tenemos en cuenta las llamadas recursivas.

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ 2T(n/2) + c_1 & \text{if } ini < fin \end{cases}$$

El coste final del algoritmo es $\mathcal{O}(n)$, siendo n el numero de elementos del vector: $n = fin - ini$.

10. Indica cuál es la recurrencia que permite calcular el coste del algoritmo quicksort si el pivote se encuentra siempre cerca de la mediana del vector (el vector se divide siempre casi por la mitad). Indica el coste final del algoritmo.

Respuesta:

$$T(n) = \begin{cases} c_0 & \text{if } ini == fin \\ 2T(n/2) + n & \text{if } ini < fin \end{cases}$$

Si el pivote se encuentra cerca de la mediana del vector, las llamadas recursivas del algoritmo del quicksort se realizan aproximadamente con la mitad de los elementos cada una. Por lo tanto tenemos dos llamadas recursivas con tamaño la mitad. El coste lineal que acompaña a las llamadas recursivas es el coste de realizar el algoritmo de partición.

El coste del algoritmo en este caso es $\mathcal{O}(n \log n)$.